

CSA0621 – DESIGN AND ANALYSIS OF ALGORITHM

LAB EXPERIMENTS

1.

main.py	Output
<pre>1 arr = [10, 25, 30, 45, 50] 2 key = 30 3 4 for i in range(len(arr)): 5 if arr[i] == key: 6 print("Key found at index", i) 7 break 8 else: 9 print("Key not found")</pre>	<pre>Key found at index 2 === Code Execution Successful ===</pre>

2.

main.py	Output
<pre>1 def binary_search(arr, low, high, key): 2 if low <= high: 3 mid = (low + high) // 2 4 if arr[mid] == key: 5 return mid 6 elif arr[mid] > key: 7 return binary_search(arr, low, mid - 1, key) 8 else: 9 return binary_search(arr, mid + 1, high, key) 10 return -1 11 12 arr = [5, 10, 15, 20, 25] 13 key = 20 14 15 result = binary_search(arr, 0, len(arr) - 1, key) 16 17 if result != -1: 18 print("Key found at index", result) 19 else: 20 print("Key not found")</pre>	<pre>Key found at index 2 === Code Execution Successful ===</pre>

3.

main.py	Output
<pre>1- def fact_recursive(n): 2- if n == 0 or n == 1: 3- return 1 4- return n * fact_recursive(n - 1) 5 6 n = 5 7 8 fact = 1 9- for i in range(1, n + 1): 10 fact *= i 11 12 print("Iterative:", fact) 13 print("Recursive:", fact_recursive(n))</pre>	<pre>Iterative: 120 Recursive: 120 === Code Execution Successful ===</pre>

4.

main.py	Output
<pre>1 n = 6 2 a, b = 0, 1 3- for i in range(n): 4 print(a, end=" ") 5 a, b = b, a + b</pre>	<pre>0 1 1 2 3 5 === Code Execution Successful ===</pre>

5.

<pre>1 A = [[1, 2], 2 [3, 4]] 3 4 B = [[5, 6], 5 [7, 8]] 6 7 result = [[0, 0], [0, 0]] 8 9- for i in range(2): 10- for j in range(2): 11- for k in range(2): 12 result[i][j] += A[i][k] * B[k][j] 13 14 print(result)</pre>	<pre>[[19, 22], [43, 50]] === Code Execution Successful ===</pre>
---	--

6.

<pre> 1- def merge_sort(a): 2- if len(a) > 1: 3- m = len(a) // 2 4- L = a[:m] 5- R = a[m:] 6- merge_sort(L) 7- merge_sort(R) 8- i = j = k = 0 9- while i < len(L) and j < len(R): 10- if L[i] < R[j]: 11- a[k] = L[i] 12- i += 1 13- else: 14- a[k] = R[j] 15- j += 1 16- k += 1 17- a[k:] = L[i:] + R[j:] 18- 19 a = [38, 27, 43, 3, 9, 82, 10] 20 merge_sort(a) 21 print(a) </pre>	<pre> [3, 9, 10, 27, 38, 43, 82] === Code Execution Successful === </pre>
--	--

7.

<pre> 1- def quick_sort(arr): 2- if len(arr) <= 1: 3- return arr 4- 5- pivot = arr[len(arr)//2] 6- left = [x for x in arr if x < pivot] 7- middle = [x for x in arr if x == pivot] 8- right = [x for x in arr if x > pivot] 9- 10- return left + middle + right 11 12 arr = [10, 7, 8, 9, 1, 5] 13 print(quick_sort(arr)) </pre>	<pre> [7, 8, 1, 5, 9, 10] === Code Execution Successful === </pre>
--	---

8.

<pre> 1- def tower(n, source, auxiliary, destination): 2- if n == 1: 3- print("Move disk 1 from", source, "to", destination) 4- return 5- tower(n - 1, source, destination, auxiliary) 6- print("Move disk", n, "from", source, "to", destination) 7- tower(n - 1, auxiliary, source, destination) 8- 9 tower(3, 'A', 'B', 'C') </pre>	<pre> Move disk 1 from A to C Move disk 2 from A to B Move disk 1 from C to B Move disk 3 from A to C Move disk 1 from B to A Move disk 2 from B to C Move disk 1 from A to C === Code Execution Successful === </pre>
--	---

9.

<pre>1 def gcd(a, b): 2 if b == 0: 3 return a 4 return gcd(b, a % b) 5 6 a = 48 7 b = 18 8 9 print("GCD =", gcd(a, b))</pre>	<pre>GCD = 6 === Code Execution Successful ===</pre>
--	--

10.

<pre>1 arr = [12, 11, 13, 5, 6] 2 3 for i in range(1, len(arr)): 4 key = arr[i] 5 j = i - 1 6 7 while j >= 0 and key < arr[j]: 8 arr[j + 1] = arr[j] 9 j -= 1 10 11 arr[j + 1] = key 12 13 print(arr)</pre>	<pre>[5, 6, 11, 12, 13] === Code Execution Successful ===</pre>
---	---

11.

<pre>1 import heapq 2 3 arr = [4, 10, 3, 5, 1] 4 heapq.heapify(arr) 5 6 while arr: 7 print(heapq.heappop(arr), end=" ")</pre>	<pre>1 3 4 5 10 === Code Execution Successful ===</pre>
---	---

12.

<pre>1 arr = [2, 4, 1, 3, 5] 2 count = 0 3 4 for i in range(len(arr)): 5 for j in range(i + 1, len(arr)): 6 if arr[i] > arr[j]: 7 count += 1 8 9 print("Inversions =", count)</pre>	<pre>Inversions = 3 === Code Execution Successful ===</pre>
--	---

13.

```
1 arr = [-2, -3, 4, -1, -2, 1, 5, -3]
2
3 max_sum = cur = arr[0]
4
5- for i in arr[1:]:
6     cur = max(i, cur + i)
7     max_sum = max(max_sum, cur)
8
9 print(max_sum)
```

7

=== Code Execution Successful ===

14.

```
1 x = 2
2 n = 10
3
4 print(x ** n)
```

1024

=== Code Execution Successful ===

15.

```
1 from math import dist
2
3 p = [(1,2), (4,5), (7,8), (3,1)]
4
5 d = float('inf')
6
7- for i in range(len(p)):
8-     for j in range(i+1, len(p)):
9         d = min(d, dist(p[i], p[j]))
10
11 print("Distance =", round(d,2))
```

Distance = 2.24

=== Code Execution Successful ===